



南開大學
Nankai University

数据结构实验报告九

红黑树

学号：2412266

姓名：杨滢

专业：信息安全

学院：密码与网络安全学院

目录

1 题目一	2
1.1 题目表述	2
1.2 代码 1 的测试用例	2
1.3 思路	3
1.3.1 继承层次设计	3
1.3.2 节点结构设计	3
1.3.3 主要操作实现	3
1.4 代码	3
1.5 测试用例截图	14
1.6 时间复杂度证明	14
1.6.1 find	14
1.6.2 insert	15
1.6.3 ascend	15
1.6.4 erase	15
2 题目二	16
2.1 题目表述	16
2.2 代码 2 的测试用例	16
2.3 思路	17
2.3.1 继承层次设计	17
2.3.2 节点结构设计	17
2.3.3 主要操作实现	17
2.4 代码	18
2.5 测试用例截图	28
2.6 时间复杂度证明	29
2.6.1 find	29
2.6.2 insert	29
2.6.3 erase	30
2.6.4 ascend	30

1 题目一

1.1 题目表述

设计一个 C++ 类 redBlackTree, 它派生于抽象类 bSTree(程序 14-1)。编写所有函数的代码并检验其正确性。函数 find、insert 和 delete 的时间复杂性必须是 $O(\log n)$, 函数 ascend 的时间复杂性应该是 $O(n)$ 。证明它们的时间复杂性。insert 和 delete 函数的实现必须采用本节的方法。

程序 14-1:

程序 14-1 C++ 抽象类 bsTree

```
template<class K, class E>
class bsTree : public dictionary<K,E>
{
    public:
        virtual void ascend() = 0;
        // 按关键字升序输出
};
```

1.2 代码 1 的测试用例

1. ascend(T) -> ()
2. find(T, 10) -> null
3. insert(T, (30, v30)), ascend(T) -> (30)
4. insert(T, (15, v15)), insert(T, (40, v40)), ascend(T) -> (15, 30, 40)
5. insert(T, (10, v10)), insert(T, (20, v20)), insert(T, (35, v35)), insert(T, (50, v50)), ascend(T) -> (10, 15, 20, 30, 35, 40, 50)
6. find(T, 20) -> (20, v20)
7. find(T, 17) -> null
8. erase(T, 10), ascend(T) -> (15, 20, 30, 35, 40, 50)
9. erase(T, 50), ascend(T) -> (15, 20, 30, 35, 40)
10. erase(T, 30), ascend(T), find(T, 30) -> (15, 20, 35, 40), null
11. insert(T, (18, v18)), insert(T, (37, v37)), ascend(T) -> (15, 18, 20, 35, 37, 40)
12. erase(T, 18), erase(T, 37), ascend(T) -> (15, 20, 35, 40)
13. clear(T), insert(T, 1), insert(T, 2), insert(T, 3), insert(T, 4), insert(T, 5), insert(T, 6), ascend(T) -> (1, 2, 3, 4, 5, 6)
14. height(T) -> 3 或者 4

1.3 思路

1.3.1 继承层次设计

最底层是字典接口类 `dictionary`，定义了字典数据结构的基本操作接口，包括判断是否为空、获取大小、查找、插入和删除。

中间层是二叉搜索树抽象类 `bSTree`，它继承自 `dictionary` 接口，并增加了一个纯虚函数 `ascend()`，用于按关键字升序输出所有元素。

最顶层是具体的 `redBlackTree` 类，它继承自 `bSTree`，实现了红黑树的所有具体算法。这种设计使得代码具有良好的扩展性，未来可以轻松实现其他类型的平衡二叉搜索树而不改变接口。

1.3.2 节点结构设计

红黑树节点 `RBTreeNode` 包含五个关键成员：键值对元素、左右孩子指针、父节点指针和节点颜色。其中父节点指针是红黑树实现的关键，它使得在插入和删除后能够沿着路径向上修复树的平衡性。键值对中的键被声明为 `const` 类型，这保证了键的不可变性，符合字典数据结构的基本要求。

1.3.3 主要操作实现

1. 查找操作 (`find`)

- (a) 时间复杂度： $O(\log n)$
- (b) 从根节点开始，根据键值比较结果沿树向下搜索
- (c) 返回指向键值对的指针，便于直接访问值

2. 插入操作 (`insert`)

- (a) 按 BST 规则找到插入位置
- (b) 创建新节点
- (c) 连接父节点
- (d) 调用 `fixInsert` 修复可能的红红冲突

3. 删除操作 (`erase`)

- (a) 找到待删节点
- (b) 无左孩子：用右孩子替代
- (c) 无右孩子：用左孩子替代
- (d) 有两个孩子：用后继节点替代
- (e) 如果被删节点为黑色，调用 `fixErase` 修复

1.4 代码

```
1 #include <iostream>
2 #include <fstream>
3 #include <string>
```

```
4 #include <sstream>
5 #include <cstdlib>
6 using namespace std;
7 template<class K, class E>
8 class dictionary {
9 public:
10     virtual ~dictionary() {}
11     virtual bool empty() const = 0;
12     virtual int size() const = 0;
13     virtual pair<const K, E>* find(const K&) const = 0;
14     virtual void insert(const pair<const K, E>&) = 0;
15     virtual void erase(const K&) = 0;
16 };
17 template<class K, class E>
18 class bSTree : public dictionary<K, E> {
19 public:
20     virtual void ascend() = 0;
21 };
22 enum Color { RED, BLACK };
23 template<class K, class E>
24 struct RBTreeNode {
25     pair<const K, E> element;
26     RBTreeNode<K, E>* leftChild;
27     RBTreeNode<K, E>* rightChild;
28     RBTreeNode<K, E>* parent;
29     Color color;
30     RBTreeNode(const pair<const K, E>& theElement,
31         RBTreeNode<K, E>* theLeft = NULL,
32         RBTreeNode<K, E>* theRight = NULL,
33         RBTreeNode<K, E>* theParent = NULL,
34         Color theColor = RED)
35         : element(theElement), leftChild(theLeft), rightChild(theRight),
36         parent(theParent), color(theColor) {}
37 };
38 template<class K, class E>
39 class redBlackTree : public bSTree<K, E> {
40 private:
41     RBTreeNode<K, E>* root;
42     int treeSize;
43     void rotateLeft(RBTreeNode<K, E>* x);
44     void rotateRight(RBTreeNode<K, E>* x);
45     void fixInsert(RBTreeNode<K, E>* x);
```

```
46 void fixErase(RBTreeNode<K, E>* x, RBTreeNode<K, E>* parent);
47 RBTreeNode<K, E>* minimum(RBTreeNode<K, E>* node) const;
48 RBTreeNode<K, E>* findNode(const K& theKey) const;
49 void clear(RBTreeNode<K, E>* node);
50 int height(RBTreeNode<K, E>* node) const;
51 void inorderWithSeparator(RBTreeNode<K, E>* node, bool& first) const;
52 public:
53     redBlackTree() : root(NULL), treeSize(0) {}
54     ~redBlackTree() { clear(); }
55     bool empty() const override { return treeSize == 0; }
56     int size() const override { return treeSize; }
57     pair<const K, E>* find(const K& theKey) const override;
58     void insert(const pair<const K, E>& thePair) override;
59     void erase(const K& theKey) override;
60     void ascend() override;
61     void clear();
62     int height() const;
63 };
64 template<class K, class E>
65 void redBlackTree<K, E>::rotateLeft(RBTreeNode<K, E>* x) {
66     RBTreeNode<K, E>* y = x->rightChild;
67     x->rightChild = y->leftChild;
68     if (y->leftChild != NULL) y->leftChild->parent = x;
69     y->parent = x->parent;
70     if (x->parent == NULL) root = y;
71     else if (x == x->parent->leftChild) x->parent->leftChild = y;
72     else x->parent->rightChild = y;
73     y->leftChild = x;
74     x->parent = y;
75 }
76 template<class K, class E>
77 void redBlackTree<K, E>::rotateRight(RBTreeNode<K, E>* y) {
78     RBTreeNode<K, E>* x = y->leftChild;
79     y->leftChild = x->rightChild;
80     if (x->rightChild != NULL) x->rightChild->parent = y;
81     x->parent = y->parent;
82     if (y->parent == NULL) root = x;
83     else if (y == y->parent->leftChild) y->parent->leftChild = x;
84     else y->parent->rightChild = x;
85     x->rightChild = y;
86     y->parent = x;
87 }
```

```
88 template<class K, class E>
89 void redBlackTree<K, E>::fixInsert(RBTreeNode<K, E>* z) {
90     while (z->parent != NULL && z->parent->color == RED) {
91         if (z->parent == z->parent->parent->leftChild) {
92             RBTreeNode<K, E>* y = z->parent->parent->rightChild;
93             if (y != NULL && y->color == RED) {
94                 z->parent->color = BLACK;
95                 y->color = BLACK;
96                 z->parent->parent->color = RED;
97                 z = z->parent->parent;
98             }
99             else {
100                 if (z == z->parent->rightChild) {
101                     z = z->parent;
102                     rotateLeft(z);
103                 }
104                 z->parent->color = BLACK;
105                 z->parent->parent->color = RED;
106                 rotateRight(z->parent->parent);
107             }
108         }
109         else {
110             RBTreeNode<K, E>* y = z->parent->parent->leftChild;
111             if (y != NULL && y->color == RED) {
112                 z->parent->color = BLACK;
113                 y->color = BLACK;
114                 z->parent->parent->color = RED;
115                 z = z->parent->parent;
116             }
117             else {
118                 if (z == z->parent->leftChild) {
119                     z = z->parent;
120                     rotateRight(z);
121                 }
122                 z->parent->color = BLACK;
123                 z->parent->parent->color = RED;
124                 rotateLeft(z->parent->parent);
125             }
126         }
127     }
128     root->color = BLACK;
129 }
```

```
130 template<class K, class E>
131 void redBlackTree<K, E>::fixErase(RBTreeNode<K, E>* x, RBTreeNode<K, E>* parent) {
132     while (x != root && (x == NULL || x->color == BLACK)) {
133         if (x == parent->leftChild) {
134             RBTreeNode<K, E>* w = parent->rightChild;
135             if (w->color == RED) {
136                 w->color = BLACK;
137                 parent->color = RED;
138                 rotateLeft(parent);
139                 w = parent->rightChild;
140             }
141             if ((w->leftChild == NULL || w->leftChild->color == BLACK) &&
142                 (w->rightChild == NULL || w->rightChild->color == BLACK)) {
143                 w->color = RED;
144                 x = parent;
145                 parent = x->parent;
146             }
147             else {
148                 if (w->rightChild == NULL || w->rightChild->color == BLACK) {
149                     if (w->leftChild != NULL) w->leftChild->color = BLACK;
150                     w->color = RED;
151                     rotateRight(w);
152                     w = parent->rightChild;
153                 }
154                 w->color = parent->color;
155                 parent->color = BLACK;
156                 if (w->rightChild != NULL) w->rightChild->color = BLACK;
157                 rotateLeft(parent);
158                 x = root;
159                 break;
160             }
161         }
162         else {
163             RBTreeNode<K, E>* w = parent->leftChild;
164             if (w->color == RED) {
165                 w->color = BLACK;
166                 parent->color = RED;
167                 rotateRight(parent);
168                 w = parent->leftChild;
169             }
170             if ((w->rightChild == NULL || w->rightChild->color == BLACK) &&
171                 (w->leftChild == NULL || w->leftChild->color == BLACK)) {
```

```

172         w->color = RED;
173         x = parent;
174         parent = x->parent;
175     }
176     else {
177         if (w->leftChild == NULL || w->leftChild->color == BLACK) {
178             if (w->rightChild != NULL) w->rightChild->color = BLACK;
179             w->color = RED;
180             rotateLeft(w);
181             w = parent->leftChild;
182         }
183         w->color = parent->color;
184         parent->color = BLACK;
185         if (w->leftChild != NULL) w->leftChild->color = BLACK;
186         rotateRight(parent);
187         x = root;
188         break;
189     }
190 }
191 }
192 if (x != NULL) x->color = BLACK;
193 }
194 template<class K, class E>
195 RBTreeNode<K, E>* redBlackTree<K, E>::findNode(const K& theKey) const {
196     RBTreeNode<K, E>* p = root;
197     while (p != NULL) {
198         if (theKey < p->element.first) p = p->leftChild;
199         else if (theKey > p->element.first) p = p->rightChild;
200         else return p;
201     }
202     return NULL;
203 }
204 template<class K, class E>
205 pair<const K, E>* redBlackTree<K, E>::find(const K& theKey) const {
206     RBTreeNode<K, E>* p = findNode(theKey);
207     return p ? &(p->element) : NULL;
208 }
209 template<class K, class E>
210 void redBlackTree<K, E>::insert(const pair<const K, E>& thePair) {
211     RBTreeNode<K, E>* p = root;
212     RBTreeNode<K, E>* pp = NULL;
213     while (p != NULL) {

```

```
214     pp = p;
215     if (thePair.first < p->element.first) p = p->leftChild;
216     else if (thePair.first > p->element.first) p = p->rightChild;
217     else {
218         p->element.second = thePair.second;
219         return;
220     }
221 }
222 RBTTreeNode<K, E>* newNode = new RBTTreeNode<K, E>(thePair, NULL, NULL, pp, RED);
223 if (root == NULL) root = newNode;
224 else if (thePair.first < pp->element.first) pp->leftChild = newNode;
225 else pp->rightChild = newNode;
226 fixInsert(newNode);
227 treeSize++;
228 }
229 template<class K, class E>
230 RBTTreeNode<K, E>* redBlackTree<K, E>::minimum(RBTTreeNode<K, E>* node) const {
231     while (node != NULL && node->leftChild != NULL)
232         node = node->leftChild;
233     return node;
234 }
235 template<class K, class E>
236 void redBlackTree<K, E>::erase(const K& theKey) {
237     RBTTreeNode<K, E>* z = findNode(theKey);
238     if (z == NULL) return;
239     RBTTreeNode<K, E>* y = z;
240     RBTTreeNode<K, E>* x = NULL;
241     RBTTreeNode<K, E>* xParent = NULL;
242     Color yOriginalColor = y->color;
243     if (z->leftChild == NULL) {
244         x = z->rightChild;
245         xParent = z->parent;
246         if (xParent == NULL) root = x;
247         else if (z == xParent->leftChild) xParent->leftChild = x;
248         else xParent->rightChild = x;
249         if (x != NULL) x->parent = xParent;
250     }
251     else if (z->rightChild == NULL) {
252         x = z->leftChild;
253         xParent = z->parent;
254         if (xParent == NULL) root = x;
255         else if (z == xParent->leftChild) xParent->leftChild = x;
```

```
256     else xParent->rightChild = x;
257     if (x != NULL) x->parent = xParent;
258 }
259 else {
260     y = minimum(z->rightChild);
261     yOriginalColor = y->color;
262     x = y->rightChild;
263     if (y->parent == z) {
264         if (x != NULL) x->parent = y;
265         xParent = y;
266     }
267     else {
268         xParent = y->parent;
269         if (xParent != NULL) {
270             if (y == xParent->leftChild) xParent->leftChild = x;
271             else xParent->rightChild = x;
272         }
273         if (x != NULL) x->parent = xParent;
274         y->rightChild = z->rightChild;
275         if (y->rightChild != NULL) y->rightChild->parent = y;
276     }
277     if (z->parent == NULL) root = y;
278     else if (z == z->parent->leftChild) z->parent->leftChild = y;
279     else z->parent->rightChild = y;
280     y->parent = z->parent;
281     y->leftChild = z->leftChild;
282     if (y->leftChild != NULL) y->leftChild->parent = y;
283     y->color = z->color;
284 }
285 delete z;
286 treeSize--;
287 if (yOriginalColor == BLACK) {
288     if (x != NULL) fixErase(x, xParent);
289     else if (xParent != NULL) fixErase(NULL, xParent);
290 }
291 }
292 template<class K, class E>
293 void redBlackTree<K, E>::inorderWithSeparator(RBTreeNode<K, E>* node, bool& first)
294     const {
295     if (node != NULL) {
296         inorderWithSeparator(node->leftChild, first);
297         if (!first) cout << ", ";
```

```
297     cout << node->element.first;
298     first = false;
299     inorderWithSeparator(node->rightChild, first);
300 }
301 }
302 template<class K, class E>
303 void redBlackTree<K, E>::ascend() {
304     cout << "(";
305     bool first = true;
306     inorderWithSeparator(root, first);
307     cout << ")" << endl;
308 }
309 template<class K, class E>
310 void redBlackTree<K, E>::clear(RBTreeNode<K, E>* node) {
311     if (node != NULL) {
312         clear(node->leftChild);
313         clear(node->rightChild);
314         delete node;
315     }
316 }
317 template<class K, class E>
318 void redBlackTree<K, E>::clear() {
319     clear(root);
320     root = NULL;
321     treeSize = 0;
322 }
323 template<class K, class E>
324 int redBlackTree<K, E>::height(RBTreeNode<K, E>* node) const {
325     if (node == NULL) return 0;
326     int leftHeight = height(node->leftChild);
327     int rightHeight = height(node->rightChild);
328     return (leftHeight > rightHeight ? leftHeight : rightHeight) + 1;
329 }
330 template<class K, class E>
331 int redBlackTree<K, E>::height() const {
332     return height(root);
333 }
334 int extractNumber(const string& str, size_t startPos) {
335     size_t i = startPos;
336     while (i < str.length() && !isdigit(str[i])) i++;
337     if (i >= str.length()) return -1;
338     size_t j = i;
```

```
339     while (j < str.length() && isdigit(str[j])) j++;
340     return stoi(str.substr(i, j - i));
341 }
342 pair<int, string> extractKeyValue(const string& str, size_t startPos) {
343     size_t i = startPos;
344     while (i < str.length() && !isdigit(str[i])) i++;
345     if (i >= str.length()) return make_pair(-1, "");
346     size_t j = i;
347     while (j < str.length() && isdigit(str[j])) j++;
348     int key = stoi(str.substr(i, j - i));
349     while (j < str.length() && str[j] != ',') j++;
350     j++;
351     while (j < str.length() && str[j] == ' ') j++;
352     size_t k = j;
353     while (k < str.length() && str[k] != ',' && str[k] != ')' && str[k] != ' ') k++;
354     return make_pair(key, str.substr(j, k - j));
355 }
356 void parseInput(const string& filename) {
357     ifstream inputFile(filename);
358     if (!inputFile) {
359         cout << "WRONG" << endl;
360         return;
361     }
362     redBlackTree<int, string> tree;
363     string line;
364     while (getline(inputFile, line)) {
365         if (line.empty()) continue;
366         if (!line.empty() && line.back() == '\\r') line.pop_back();
367         size_t start = 0;
368         while (start < line.length()) {
369             while (start < line.length() && line[start] == ' ') start++;
370             if (start >= line.length()) break;
371             if (line.find("ascend(T)", start) == start) {
372                 cout << "ascend(T) -> ";
373                 tree.ascend();
374                 start += 9;
375             }
376             else if (line.find("find(T,", start) == start) {
377                 int key = extractNumber(line, start + 7);
378                 if (key != -1) {
379                     auto result = tree.find(key);
380                     cout << "find(T, " << key << ") -> ";
```

```
381         if (!result) cout << "null" << endl;
382         else cout << "(" << result->first << ", " << result->second << ")"
           << endl;
383     }
384     size_t endPos = line.find(')', start);
385     start = (endPos != string::npos) ? endPos + 1 : line.length();
386 }
387 else if (line.find("insert(T, (", start) == start) {
388     auto kv = extractKeyValue(line, start + 11);
389     if (kv.first != -1) {
390         tree.insert(kv);
391         cout << "insert(T, (" << kv.first << ", " << kv.second << "))" <<
           endl;
392     }
393     size_t endPos = line.find(')', start);
394     start = (endPos != string::npos) ? endPos + 1 : line.length();
395 }
396 else if (line.find("insert(T,", start) == start) {
397     int key = extractNumber(line, start + 9);
398     if (key != -1) {
399         tree.insert(make_pair(key, "v" + to_string(key)));
400         cout << "insert(T, " << key << ")" << endl;
401     }
402     size_t endPos = line.find(')', start);
403     start = (endPos != string::npos) ? endPos + 1 : line.length();
404 }
405 else if (line.find("erase(T,", start) == start) {
406     int key = extractNumber(line, start + 8);
407     if (key != -1) {
408         tree.erase(key);
409         cout << "erase(T, " << key << ")" << endl;
410     }
411     size_t endPos = line.find(')', start);
412     start = (endPos != string::npos) ? endPos + 1 : line.length();
413 }
414 else if (line.find("clear(T)", start) == start) {
415     tree.clear();
416     cout << "clear(T)" << endl;
417     start += 8;
418 }
419 else if (line.find("height(T)", start) == start) {
420     cout << "height(T) -> " << tree.height() << endl;
```

```

421         start += 9;
422     }
423     else {
424         start++;
425     }
426 }
427 }
428 inputFile.close();
429 }
430 int main() {
431     parseInput("input.txt");
432     system("pause");
433     return 0;
434 }

```

1.5 测试用例截图

```

ascend(T) -> ()
find(T, 10) -> null
insert(T, (30, v30))
ascend(T) -> (30)
insert(T, (15, v15))
insert(T, (40, v40))
ascend(T) -> (15, 30, 40)
insert(T, (10, v10))
insert(T, (20, v20))
insert(T, (35, v35))
insert(T, (50, v50))
ascend(T) -> (10, 15, 20, 30, 35, 40, 50)
find(T, 20) -> (20, v20)
find(T, 17) -> null
erase(T, 10)
ascend(T) -> (15, 20, 30, 35, 40, 50)
erase(T, 50)
ascend(T) -> (15, 20, 30, 35, 40)
erase(T, 30)
ascend(T) -> (15, 20, 35, 40)
find(T, 30) -> null
insert(T, (18, v18))
insert(T, (37, v37))
ascend(T) -> (15, 18, 20, 35, 37, 40)
erase(T, 18)
erase(T, 37)
ascend(T) -> (15, 20, 35, 40)
clear(T)
insert(T, 1)
insert(T, 2)
insert(T, 3)
insert(T, 4)
insert(T, 5)
insert(T, 6)
ascend(T) -> (1, 2, 3, 4, 5, 6)
height(T) -> 4
请按任意键继续...

```

1.6 时间复杂度证明

1.6.1 find

- 查找过程从根节点开始
- 每次比较后，根据比较结果选择左子树或右子树
- 由于红黑树是平衡的，树的高度 $h \leq 2\log(n+1)$

在最坏情况下需要遍历树的高度。因此，查找的时间复杂度为 $O(h) = O(\log n)$

1.6.2 insert

1. 插入操作分为两个阶段：

- 查找插入位置： $O(\log n)$
- 修复红黑树性质（旋转和重新着色）： $O(\log n)$

2. 查找阶段：

- 从根节点开始向下查找插入位置
- 最多遍历树的高度 $h = O(\log n)$ 个节点

3. 修复阶段：

- 插入新节点后，最多需要进行：2 次旋转（左旋/右旋）。重新着色操作向上传播，最多传播到根节点
- 旋转操作本身是 $O(1)$
- 重新着色最多进行 $O(\log n)$ 次（沿着路径向上）

得到总时间复杂度： $O(\log n) + O(\log n) = O(\log n)$

1.6.3 ascend

- `ascend()` 使用中序遍历实现
- 中序遍历访问每个节点恰好一次
- 访问每个节点的操作是 $O(1)$

对于 n 个节点的树，需要访问 n 次。因此，时间复杂度为 $O(n)$ 。

1.6.4 erase

1. 删除操作分为三个阶段：

- 查找要删除的节点： $O(\log n)$
- 执行删除操作： $O(1)$ 或 $O(\log n)$
- 修复红黑树性质： $O(\log n)$

2. 查找阶段： $O(\log n)$

- 从根节点开始查找要删除的节点
- 最多遍历树的高度 h 个节点，其中 $h = O(\log n)$

3. 删除阶段：

- 情况 1：删除的节点有 0 或 1 个子节点 $\Rightarrow O(1)$
- 情况 2：删除的节点有 2 个子节点 $\Rightarrow$ 需要找到后继节点 $O(\log n)$

4. 修复阶段：

- 最多进行 3 次旋转
- 重新着色操作向上传播，最多传播到根节点
- 传播路径长度 $\leq h = O(\log n)$

得到总时间复杂度： $O(\log n) + O(\log n) = O(\log n)$

2 题目二

2.1 题目表述

设计一个 C++ 类 dRedBlackTree, 它派生于抽象类 dBSTree(参看练习 4)。编写所有函数的代码并检验其正确性。函数 find、insert 和 delete 必须具有复杂性 $O(\log n)$, 函数 ascend 的时间复杂性应该是 $O(n)$ 。证明它们的时间复杂性。

2.2 代码 2 的测试用例

1. ascend(T) -> ()
2. find(T, 10) -> null
3. insert(T, (20, v20)), insert(T, (10, v10)), insert(T, (30, v30)), ascend(T) -> (10, 20, 30)
4. insert(T, (10, v10_2)), insert(T, (10, v10_3)), ascend(T) -> (10, 10, 10, 20, 30)
5. find(T, 10) -> (10, v?)
6. erase(T, 10), ascend(T) -> (10, 10, 20, 30)
7. erase(T, 10), erase(T, 10), ascend(T) -> (20, 30)
8. find(T, 10) -> null
9. clear(T), insert(T, (5, A)), insert(T, (5, B)), insert(T, (5, C)), insert(T, (3, D)), insert(T, (8, E)), ascend(T) -> (3, 5, 5, 5, 8)
10. erase(T, 5), ascend(T) -> (3, 5, 5, 8)
11. clear(T), insert(T, 1), insert(T, 2), insert(T, 3), insert(T, 4), insert(T, 5), insert(T, 6), ascend(T) -> (1, 2, 3, 4, 5, 6)
12. height(T) -> 3 或者 4
13. clear(T), insert(T, 50), insert(T, 40), insert(T, 30), insert(T, 20), insert(T, 10), ascend(T) -> (10, 20, 30, 40, 50)
14. height(T) -> 3 或者 4

2.3 思路

2.3.1 继承层次设计

本系统采用三层递进的面向对象继承结构，构建了一个模块化、可扩展的字典容器框架。最底层是抽象字典接口类 `dictionary`，采用纯虚函数定义了字典数据结构的标准操作契约，包括判断容器是否为空、获取元素数量、根据键查找值、插入新键值对以及删除指定键值对等基本功能，为所有字典实现提供了统一的编程接口。

中间层是二叉搜索树抽象类 `dBSTree`，它继承自 `dictionary` 接口，在保留字典基本功能的基础上，新增了一个纯虚函数 `ascend()`，专门用于按关键字升序输出树中的所有元素，这一层为各种二叉搜索树实现提供了共同的抽象基类。

最顶层是具体的红黑树实现类 `dRedBlackTree`，它继承自 `dBSTree`，完整实现了红黑树的自平衡算法，包括节点的旋转操作、插入后的修复逻辑以及删除后的平衡调整等核心机制。

2.3.2 节点结构设计

- **dRBTreeNode 结构体**包含五个关键成员：

1. `pair<const K, E> element`：键值对元素
 - 键声明为 `const` 类型，保证键的不可变性
 - 符合字典数据结构的基本要求
2. `dRBTreeNode* leftChild`：左子树指针
3. `dRBTreeNode* rightChild`：右子树指针
4. `dRBTreeNode* parent`：父节点指针
 - 红黑树实现的关键特性
 - 支持从下向上的修复操作
 - 便于实现旋转操作时的指针更新
5. `Color color`：节点颜色（RED/BLACK）
 - 枚举类型：RED=0, BLACK=1
 - 新插入节点默认颜色为 RED

2.3.3 主要操作实现

1. 查找操作 (`find`)

- (a) 时间复杂度： $O(\log n)$
- (b) 从根节点开始，根据键值比较结果沿树向下搜索
- (c) 允许重复键：找到第一个匹配节点即返回
- (d) 返回指向键值对的指针，便于直接访问值
- (e) 查找失败返回 `NULL`

2. 插入操作 (`insert`)

- (a) 时间复杂度： $O(\log n)$

- (b) 允许重复键：相等键值插入到右子树
- (c) 按 BST 规则找到插入位置
- (d) 创建新节点（初始颜色为 RED）
- (e) 连接父节点，建立父子关系
- (f) 调用 `fixInsert` 修复可能的红红冲突

3. 删除操作 (erase)

- (a) 时间复杂度： $O(\log n)$
- (b) 找到待删节点（仅删除第一个匹配的重复键）
- (c) 分三种情况处理：
 - i. 无左孩子：用右孩子替代
 - ii. 无右孩子：用左孩子替代
 - iii. 有两个孩子：用后继节点替代
- (d) 如果被删节点为黑色，调用 `fixDelete` 修复
- (e) 释放被删节点内存，更新 `treeSize`

4. 遍历操作 (ascend)

- (a) 时间复杂度： $O(n)$
- (b) 使用中序遍历实现升序输出
- (c) 输出格式：`(key1, key2, ..., keyN)`
- (d) 逗号分隔，包含所有重复键
- (e) 递归实现，使用辅助函数 `inorderWithSeparator`

2.4 代码

```
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4 #include <sstream>
5 #include <cstdlib>
6 using namespace std;
7 template<class K, class E>
8 class dictionary {
9 public:
10     virtual ~dictionary() {}
11     virtual bool empty() const = 0;
12     virtual int size() const = 0;
13     virtual pair<const K, E>* find(const K&) const = 0;
14     virtual void insert(const pair<const K, E>&) = 0;
```

```
15     virtual void erase(const K&) = 0;
16 };
17 template<class K, class E>
18 class dBSTree : public dictionary<K, E> {
19 public:
20     virtual void ascend() = 0;
21 };
22 enum Color { RED, BLACK };
23 template<class K, class E>
24 struct dRBTreeNode {
25     pair<const K, E> element;
26     dRBTreeNode<K, E>* leftChild;
27     dRBTreeNode<K, E>* rightChild;
28     dRBTreeNode<K, E>* parent;
29     Color color;
30     dRBTreeNode(const pair<const K, E>& theElement,
31                 dRBTreeNode<K, E>* theLeft = NULL,
32                 dRBTreeNode<K, E>* theRight = NULL,
33                 dRBTreeNode<K, E>* theParent = NULL,
34                 Color theColor = RED)
35         : element(theElement), leftChild(theLeft), rightChild(theRight),
36           parent(theParent), color(theColor) {}
37 };
38
39 template<class K, class E>
40 class dRedBlackTree : public dBSTree<K, E> {
41 private:
42     dRBTreeNode<K, E>* root;
43     int treeSize;
44     void rotateLeft(dRBTreeNode<K, E>* x) {
45         dRBTreeNode<K, E>* y = x->rightChild;
46         x->rightChild = y->leftChild;
47         if (y->leftChild != NULL) y->leftChild->parent = x;
48         y->parent = x->parent;
49         if (x->parent == NULL) root = y;
50         else if (x == x->parent->leftChild) x->parent->leftChild = y;
51         else x->parent->rightChild = y;
52         y->leftChild = x;
53         x->parent = y;
54     }
55     void rotateRight(dRBTreeNode<K, E>* y) {
56         dRBTreeNode<K, E>* x = y->leftChild;
```

```
57     y->leftChild = x->rightChild;
58     if (x->rightChild != NULL) x->rightChild->parent = y;
59     x->parent = y->parent;
60     if (y->parent == NULL) root = x;
61     else if (y == y->parent->leftChild) y->parent->leftChild = x;
62     else y->parent->rightChild = x;
63     x->rightChild = y;
64     y->parent = x;
65 }
66 void fixInsert(dRBTreeNode<K, E>* z) {
67     while (z->parent != NULL && z->parent->color == RED) {
68         if (z->parent == z->parent->parent->leftChild) {
69             dRBTreeNode<K, E>* y = z->parent->parent->rightChild;
70             if (y != NULL && y->color == RED) {
71                 z->parent->color = BLACK;
72                 y->color = BLACK;
73                 z->parent->parent->color = RED;
74                 z = z->parent->parent;
75             }
76             else {
77                 if (z == z->parent->rightChild) {
78                     z = z->parent;
79                     rotateLeft(z);
80                 }
81                 z->parent->color = BLACK;
82                 z->parent->parent->color = RED;
83                 rotateRight(z->parent->parent);
84             }
85         }
86         else {
87             dRBTreeNode<K, E>* y = z->parent->parent->leftChild;
88             if (y != NULL && y->color == RED) {
89                 z->parent->color = BLACK;
90                 y->color = BLACK;
91                 z->parent->parent->color = RED;
92                 z = z->parent->parent;
93             }
94             else {
95                 if (z == z->parent->leftChild) {
96                     z = z->parent;
97                     rotateRight(z);
98                 }
99             }
100         }
101     }
102 }
```

```
99         z->parent->color = BLACK;
100        z->parent->parent->color = RED;
101        rotateLeft(z->parent->parent);
102    }
103    }
104    }
105    root->color = BLACK;
106    }
107    void fixDelete(dRBTreeNode<K, E>* x, dRBTreeNode<K, E>* parent) {
108        while (x != root && (x == NULL || x->color == BLACK)) {
109            if (x == parent->leftChild) {
110                dRBTreeNode<K, E>* w = parent->rightChild;
111                if (w->color == RED) {
112                    w->color = BLACK;
113                    parent->color = RED;
114                    rotateLeft(parent);
115                    w = parent->rightChild;
116                }
117                if ((w->leftChild == NULL || w->leftChild->color == BLACK) &&
118                    (w->rightChild == NULL || w->rightChild->color == BLACK)) {
119                    w->color = RED;
120                    x = parent;
121                    parent = x->parent;
122                }
123                else {
124                    if (w->rightChild == NULL || w->rightChild->color == BLACK) {
125                        if (w->leftChild != NULL) w->leftChild->color = BLACK;
126                        w->color = RED;
127                        rotateRight(w);
128                        w = parent->rightChild;
129                    }
130                    w->color = parent->color;
131                    parent->color = BLACK;
132                    if (w->rightChild != NULL) w->rightChild->color = BLACK;
133                    rotateLeft(parent);
134                    x = root;
135                    break;
136                }
137            }
138            else {
139                dRBTreeNode<K, E>* w = parent->leftChild;
140                if (w->color == RED) {
```

```
141         w->color = BLACK;
142         parent->color = RED;
143         rotateRight(parent);
144         w = parent->leftChild;
145     }
146     if ((w->rightChild == NULL || w->rightChild->color == BLACK) &&
147         (w->leftChild == NULL || w->leftChild->color == BLACK)) {
148         w->color = RED;
149         x = parent;
150         parent = x->parent;
151     }
152     else {
153         if (w->leftChild == NULL || w->leftChild->color == BLACK) {
154             if (w->rightChild != NULL) w->rightChild->color = BLACK;
155             w->color = RED;
156             rotateLeft(w);
157             w = parent->leftChild;
158         }
159         w->color = parent->color;
160         parent->color = BLACK;
161         if (w->leftChild != NULL) w->leftChild->color = BLACK;
162         rotateRight(parent);
163         x = root;
164         break;
165     }
166 }
167 }
168 if (x != NULL) x->color = BLACK;
169 }
170 dRBTreeNode<K, E>* minimum(dRBTreeNode<K, E>* node) const {
171     while (node != NULL && node->leftChild != NULL)
172         node = node->leftChild;
173     return node;
174 }
175 dRBTreeNode<K, E>* findNode(const K& theKey) const {
176     dRBTreeNode<K, E>* p = root;
177     while (p != NULL) {
178         if (theKey < p->element.first) p = p->leftChild;
179         else if (theKey > p->element.first) p = p->rightChild;
180         else return p;
181     }
182     return NULL;
```

```
183     }
184     void clear(dRBTreeNode<K, E>* node) {
185         if (node != NULL) {
186             clear(node->leftChild);
187             clear(node->rightChild);
188             delete node;
189         }
190     }
191     int height(dRBTreeNode<K, E>* node) const {
192         if (node == NULL) return 0;
193         int leftHeight = height(node->leftChild);
194         int rightHeight = height(node->rightChild);
195         return (leftHeight > rightHeight ? leftHeight : rightHeight) + 1;
196     }
197     void inorderWithSeparator(dRBTreeNode<K, E>* node, bool& first) const {
198         if (node != NULL) {
199             inorderWithSeparator(node->leftChild, first);
200             if (!first) cout << ", ";
201             cout << node->element.first;
202             first = false;
203             inorderWithSeparator(node->rightChild, first);
204         }
205     }
206 public:
207     dRedBlackTree() : root(NULL), treeSize(0) {}
208     ~dRedBlackTree() { clear(); }
209     bool empty() const override { return treeSize == 0; }
210     int size() const override { return treeSize; }
211     pair<const K, E>* find(const K& theKey) const override {
212         dRBTreeNode<K, E>* p = findNode(theKey);
213         return p ? &(p->element) : NULL;
214     }
215     void insert(const pair<const K, E>& thePair) override {
216         dRBTreeNode<K, E>* p = root;
217         dRBTreeNode<K, E>* pp = NULL;
218         while (p != NULL) {
219             pp = p;
220             if (thePair.first < p->element.first) p = p->leftChild;
221             else p = p->rightChild;
222         }
223         dRBTreeNode<K, E>* newNode = new dRBTreeNode<K, E>(thePair, NULL, NULL, pp,
224             RED);
```

```
224     if (root == NULL) root = newNode;
225     else if (thePair.first < pp->element.first) pp->leftChild = newNode;
226     else pp->rightChild = newNode;
227     fixInsert(newNode);
228     treeSize++;
229 }
230 void erase(const K& theKey) override {
231     dRBTTreeNode<K, E>* z = findNode(theKey);
232     if (z == NULL) return;
233     dRBTTreeNode<K, E>* y = z;
234     dRBTTreeNode<K, E>* x = NULL;
235     dRBTTreeNode<K, E>* xParent = NULL;
236     Color yOriginalColor = y->color;
237     if (z->leftChild == NULL) {
238         x = z->rightChild;
239         xParent = z->parent;
240         if (xParent == NULL) root = x;
241         else if (z == xParent->leftChild) xParent->leftChild = x;
242         else xParent->rightChild = x;
243         if (x != NULL) x->parent = xParent;
244     }
245     else if (z->rightChild == NULL) {
246         x = z->leftChild;
247         xParent = z->parent;
248         if (xParent == NULL) root = x;
249         else if (z == xParent->leftChild) xParent->leftChild = x;
250         else xParent->rightChild = x;
251         if (x != NULL) x->parent = xParent;
252     }
253     else {
254         y = minimum(z->rightChild);
255         yOriginalColor = y->color;
256         x = y->rightChild;
257         if (y->parent == z) {
258             if (x != NULL) x->parent = y;
259             xParent = y;
260         }
261         else {
262             xParent = y->parent;
263             if (xParent != NULL) {
264                 if (y == xParent->leftChild) xParent->leftChild = x;
265                 else xParent->rightChild = x;
```

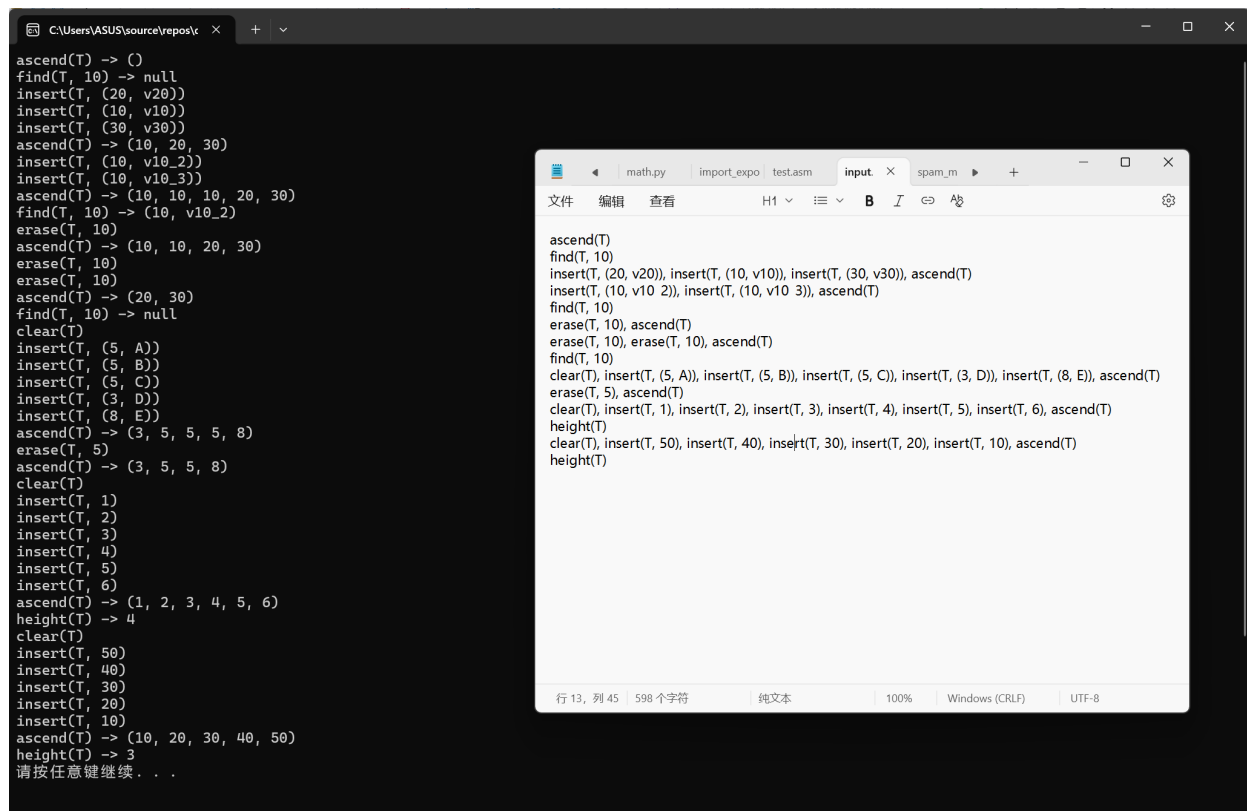
```
266         }
267         if (x != NULL) x->parent = xParent;
268         y->rightChild = z->rightChild;
269         if (y->rightChild != NULL) y->rightChild->parent = y;
270     }
271     if (z->parent == NULL) root = y;
272     else if (z == z->parent->leftChild) z->parent->leftChild = y;
273     else z->parent->rightChild = y;
274     y->parent = z->parent;
275     y->leftChild = z->leftChild;
276     if (y->leftChild != NULL) y->leftChild->parent = y;
277     y->color = z->color;
278 }
279 delete z;
280 treeSize--;
281 if (yOriginalColor == BLACK) {
282     if (x != NULL) fixDelete(x, xParent);
283     else if (xParent != NULL) fixDelete(NULL, xParent);
284 }
285 }
286 void ascend() override {
287     cout << "(";
288     bool first = true;
289     inorderWithSeparator(root, first);
290     cout << ")" << endl;
291 }
292 void clear() {
293     clear(root);
294     root = NULL;
295     treeSize = 0;
296 }
297 int height() const {
298     return height(root);
299 }
300 };
301 int extractNumber(const string& str, size_t startPos) {
302     size_t i = startPos;
303     while (i < str.length() && !isdigit(str[i])) i++;
304     if (i >= str.length()) return -1;
305     size_t j = i;
306     while (j < str.length() && isdigit(str[j])) j++;
307     return stoi(str.substr(i, j - i));
```

```
308 }
309
310 pair<int, string> extractKeyValue(const string& str, size_t startPos) {
311     size_t i = startPos;
312     while (i < str.length() && !isdigit(str[i])) i++;
313     if (i >= str.length()) return make_pair(-1, "");
314     size_t j = i;
315     while (j < str.length() && isdigit(str[j])) j++;
316     int key = stoi(str.substr(i, j - i));
317     while (j < str.length() && str[j] != ',') j++;
318     j++;
319     while (j < str.length() && str[j] == ' ') j++;
320     size_t k = j;
321     while (k < str.length() && str[k] != ',' && str[k] != ')' && str[k] != ' ') k++;
322     return make_pair(key, str.substr(j, k - j));
323 }
324
325 void parseInput(const string& filename) {
326     ifstream inputFile(filename);
327     if (!inputFile) {
328         cout << "WRONG" << endl;
329         return;
330     }
331     dRedBlackTree<int, string> tree;
332     string line;
333     while (getline(inputFile, line)) {
334         if (line.empty()) continue;
335         if (!line.empty() && line.back() == '\\r') line.pop_back();
336         size_t start = 0;
337         while (start < line.length()) {
338             while (start < line.length() && line[start] == ' ') start++;
339             if (start >= line.length()) break;
340             if (line.find("ascend(T)", start) == start) {
341                 cout << "ascend(T) -> ";
342                 tree.ascend();
343                 start += 9;
344             }
345             else if (line.find("find(T,", start) == start) {
346                 int key = extractNumber(line, start + 7);
347                 if (key != -1) {
348                     auto result = tree.find(key);
349                     cout << "find(T, " << key << ") -> ";
```

```
350         if (!result) cout << "null" << endl;
351         else cout << "(" << result->first << ", " << result->second << ")"
           << endl;
352     }
353     size_t endPos = line.find(')', start);
354     start = (endPos != string::npos) ? endPos + 1 : line.length();
355 }
356 else if (line.find("insert(T, (", start) == start) {
357     auto kv = extractKeyValue(line, start + 11);
358     if (kv.first != -1) {
359         tree.insert(kv);
360         cout << "insert(T, (" << kv.first << ", " << kv.second << "))" <<
           endl;
361     }
362     size_t endPos = line.find(')', start);
363     start = (endPos != string::npos) ? endPos + 1 : line.length();
364 }
365 else if (line.find("insert(T,", start) == start) {
366     int key = extractNumber(line, start + 9);
367     if (key != -1) {
368         tree.insert(make_pair(key, "v" + to_string(key)));
369         cout << "insert(T, " << key << ")" << endl;
370     }
371     size_t endPos = line.find(')', start);
372     start = (endPos != string::npos) ? endPos + 1 : line.length();
373 }
374 else if (line.find("erase(T,", start) == start) {
375     int key = extractNumber(line, start + 8);
376     if (key != -1) {
377         tree.erase(key);
378         cout << "erase(T, " << key << ")" << endl;
379     }
380     size_t endPos = line.find(')', start);
381     start = (endPos != string::npos) ? endPos + 1 : line.length();
382 }
383 else if (line.find("clear(T)", start) == start) {
384     tree.clear();
385     cout << "clear(T)" << endl;
386     start += 8;
387 }
388 else if (line.find("height(T)", start) == start) {
389     cout << "height(T) -> " << tree.height() << endl;
```

```
390         start += 9;
391     }
392     else {
393         start++;
394     }
395 }
396 }
397 inputFile.close();
398 }
399
400 int main() {
401     parseInput("input.txt");
402     system("pause");
403     return 0;
404 }
```

2.5 测试用例截图



```
ascend(T) -> ()
find(T, 10) -> null
insert(T, (20, v20))
insert(T, (10, v10))
insert(T, (30, v30))
ascend(T) -> (10, 20, 30)
insert(T, (10, v10_2))
insert(T, (10, v10_3))
ascend(T) -> (10, 10, 10, 20, 30)
find(T, 10) -> (10, v10_2)
erase(T, 10)
ascend(T) -> (10, 10, 20, 30)
erase(T, 10)
erase(T, 10)
ascend(T) -> (20, 30)
find(T, 10) -> null
clear(T)
insert(T, (5, A))
insert(T, (5, B))
insert(T, (5, C))
insert(T, (3, D))
insert(T, (8, E))
ascend(T) -> (3, 5, 5, 5, 8)
erase(T, 5)
ascend(T) -> (3, 5, 5, 8)
clear(T)
insert(T, 1)
insert(T, 2)
insert(T, 3)
insert(T, 4)
insert(T, 5)
insert(T, 6)
ascend(T) -> (1, 2, 3, 4, 5, 6)
height(T) -> 4
clear(T)
insert(T, 50)
insert(T, 40)
insert(T, 30)
insert(T, 20)
insert(T, 10)
ascend(T) -> (10, 20, 30, 40, 50)
height(T) -> 3
请按任意键继续...
```

2.6 时间复杂度证明

2.6.1 find

- 查找过程：
 - 从根节点开始，每次比较键值大小
 - 根据比较结果选择进入左子树或右子树
 - 最多遍历树的高度 h 个节点
- 红黑树平衡性：
 - 红黑树是近似平衡的二叉搜索树
 - 树的高度满足： $h \leq 2\log_2(n+1)$
 - 最坏情况下树高为 $O(\log n)$
- 时间复杂度推导：

$$T_{\text{find}}(n) = h \times O(1) = O(h) = O(2\log_2(n+1)) = O(\log n)$$

其中每次节点访问操作时间为常数 $O(1)$

2.6.2 insert

1. 操作分解：
 - 阶段一：查找插入位置 $O(\log n)$
 - 阶段二：修复红黑树性质 $O(\log n)$
2. 查找阶段分析：
 - 从根节点向下查找合适插入位置
 - 最多遍历 $h = O(\log n)$ 个节点
 - 每层比较时间为 $O(1)$
3. 修复阶段分析：
 - `fixInsert` 函数最多进行：
 - 从插入点向上遍历到根：最多 h 次迭代
 - 每次迭代最多执行：
 - * 1 次旋转操作： $O(1)$
 - * 重新着色操作： $O(1)$
 - 修复操作总时间： $h \times O(1) = O(\log n)$
4. 总时间复杂度：

$$T_{\text{insert}}(n) = O(\log n) + O(\log n) = O(\log n)$$

2.6.3 erase

1. 操作分解:

- 阶段一: 查找要删除的节点 $O(\log n)$
- 阶段二: 执行删除操作 $O(1)$ 或 $O(\log n)$
- 阶段三: 修复红黑树性质 $O(\log n)$

2. 查找阶段:

$$T_{\text{search}} = h \times O(1) = O(\log n)$$

3. 删除阶段 (分情况分析):

- 情况 1: 节点有 0 或 1 个子节点
 - 直接替换: $O(1)$
- 情况 2: 节点有 2 个子节点
 - 查找后继节点: 从右子树向左遍历
 - 最多遍历 $h = O(\log n)$ 个节点
 - 删除操作时间: $O(\log n)$

4. 修复阶段分析:

- 函数最多进行:
 - 从删除点向上遍历到根: 最多 h 次迭代
 - 每次迭代最多执行:
 - * 1 次旋转操作: $O(1)$
 - * 重新着色操作: $O(1)$
- 修复操作总时间: $h \times O(1) = O(\log n)$

5. 最坏情况分析:

- 删除有 2 个子节点的节点
- 需要查找后继节点
- 总时间 = 查找 ($O(\log n)$) + 删除 ($O(\log n)$) + 修复 ($O(\log n)$)

6. 总时间复杂度:

$$T_{\text{erase}}(n) = O(\log n) + O(\log n) + O(\log n) = O(\log n)$$

2.6.4 ascend

• 算法实现:

- 使用中序遍历 (inorder traversal)
- 递归访问左子树 → 访问当前节点 → 递归访问右子树

• 访问节点次数分析:

- 树中有 n 个节点
- 每个节点恰好被访问一次
- 访问每个节点的操作：
 - * 输出键值: $O(1)$
 - * 处理分隔符: $O(1)$
 - * 递归调用: $O(1)$ (函数调用开销)

• 数学归纳法证明:

设 $T(n)$ 表示遍历 n 个节点的时间

基本情况: $n = 0$ (空树)

$$T(0) = O(1)$$

归纳假设: 对于所有 $k < n$, 有 $T(k) = c \times k$ (c 为常数)

归纳步骤: 对于 n 个节点的树

- 左子树节点数: m
- 右子树节点数: $n - m - 1$
- 当前节点访问时间: d (常数)

$$\begin{aligned} T(n) &= T(m) + d + T(n - m - 1) \\ &= c \times m + d + c \times (n - m - 1) \\ &= c \times n + (d - c) \\ &= O(n) \end{aligned}$$

• 关键结论:

- 无论树的结构如何 (平衡或退化为链表)
- 中序遍历必须访问所有 n 个节点
- 每个节点的访问时间是常数
- 总时间与节点数成正比: $T(n) = O(n)$

• 时间表达式:

$$T_{\text{ascend}}(n) = n \times O(1) = O(n)$$